

Guía práctica:

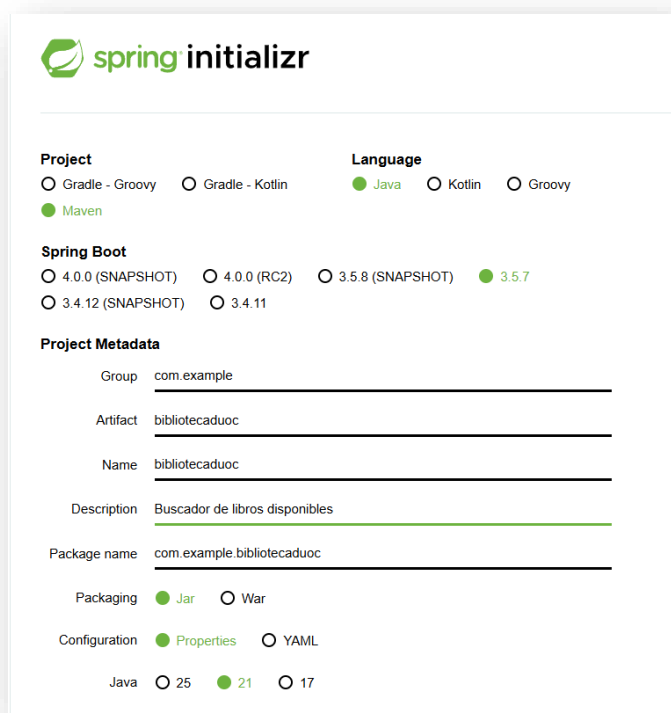
Bibliotecas Duoc – Inventario de libros

En esta guía, detallaremos paso a paso el desarrollo de un proyecto en Spring llamado "BibliotecaDuoc".

A continuación, se describen los pasos para crear tu primer proyecto en Spring Boot.

Paso 1: Ir a Spring Initializr

- Abre tu navegador web y ve a la siguiente página: [Spring Initializr](https://start.spring.io/). (<https://start.spring.io/>)

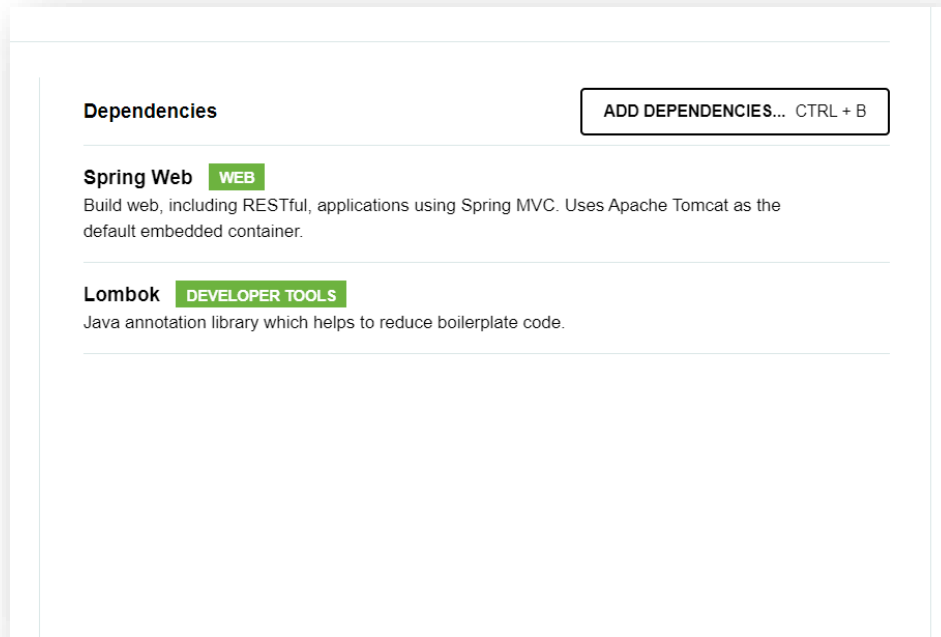


The screenshot shows the Spring Initializr web form. It includes sections for Project, Language, Spring Boot, Project Metadata, and Packaging. The form is pre-filled with the following values:

- Project:** ☒ Maven
- Language:** ☒ Java
- Spring Boot:** ☒ 3.5.7
- Project Metadata:**
 - Group: com.example
 - Artifact: bibliotecaduoc
 - Name: bibliotecaduoc
 - Description: Buscador de libros disponibles
 - Package name: com.example.bibliotecaduoc
- Packaging:** ☒ Jar
- Configuration:** ☒ Properties
- Java:** ☒ 21

Paso 2: Añadir Dependencias

- Haz clic en el botón "Add Dependencies" y añade la siguiente dependencia:
 - **Spring Web:** Para crear aplicaciones web, incluyendo RESTful.
 - **Lombok:**
 - Es una biblioteca de Java que se utiliza para **reducir el código repetitivo y redundante** mediante el uso de anotaciones. Simplifica el desarrollo al generar automáticamente el código boilerplate, como getters, setters, constructores, métodos toString, equals, hashCode, entre otros, durante la compilación. Esto mejora la legibilidad y mantenibilidad del código.

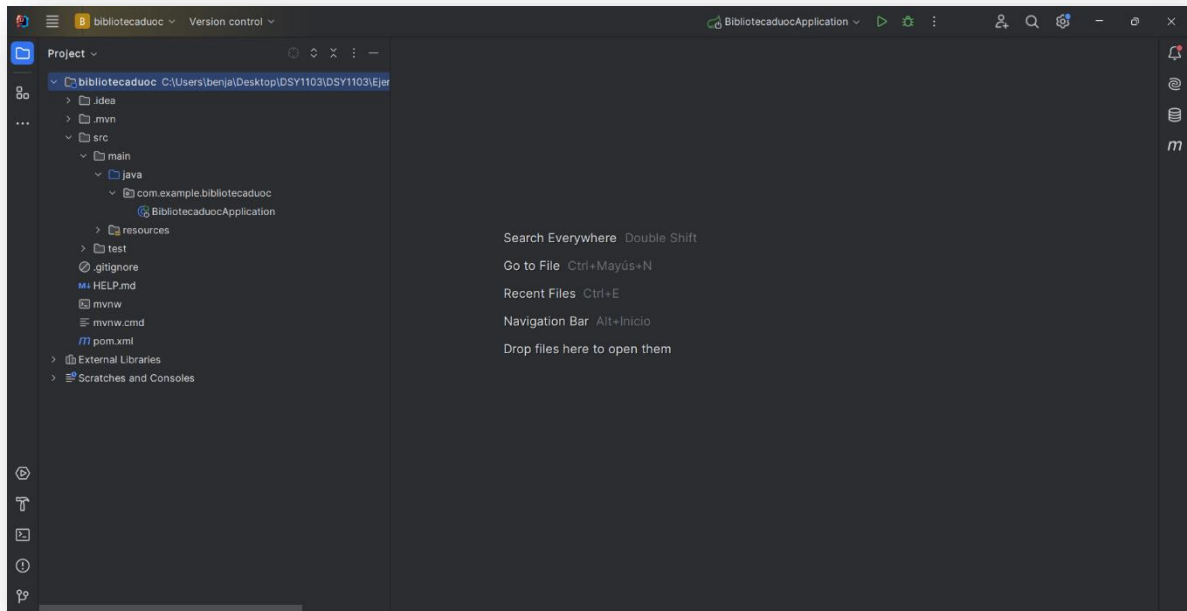


Paso 3: Generar el Proyecto

- Una vez que hayas configurado todo, haz clic en el botón "**GENERATE**" para descargar un archivo ZIP con tu proyecto Spring Boot.

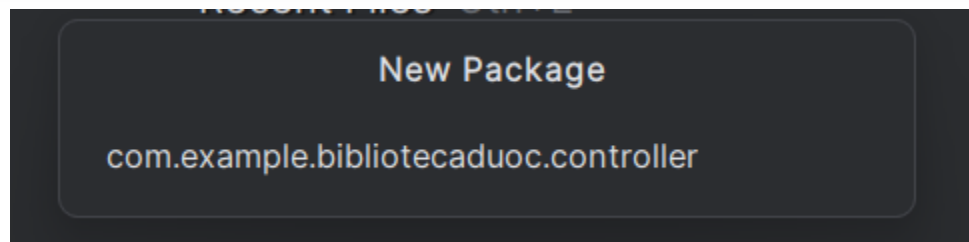
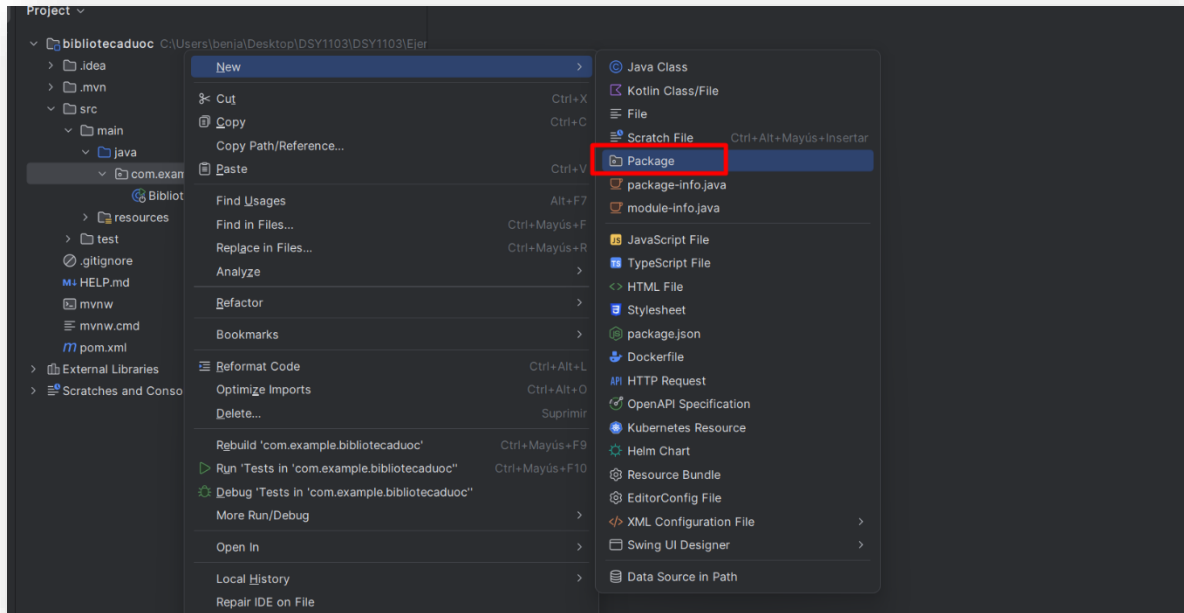
Paso 4: Importar el Proyecto en tu IDE

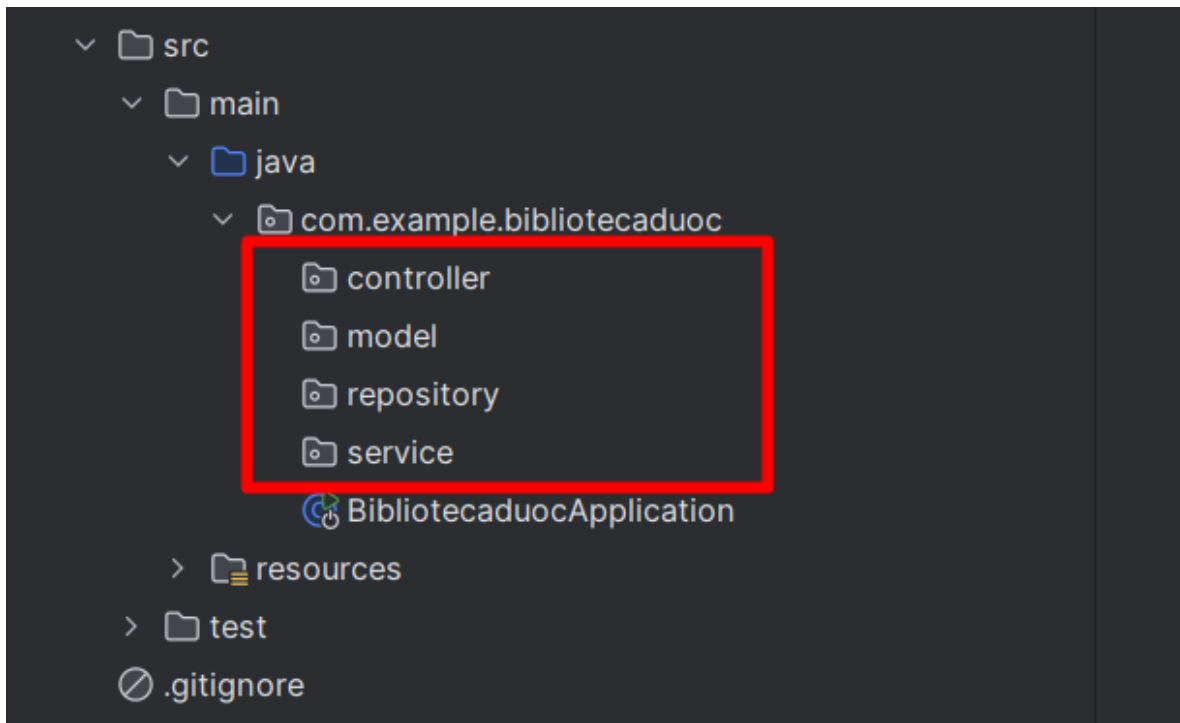
- Descomprime el archivo ZIP descargado y abre tu IDE preferido (VS Code o IntelliJ IDEA).
 - Navega hasta la carpeta descomprimida y selecciona el proyecto para importarlo.



Paso 5: Ordenando carpetas

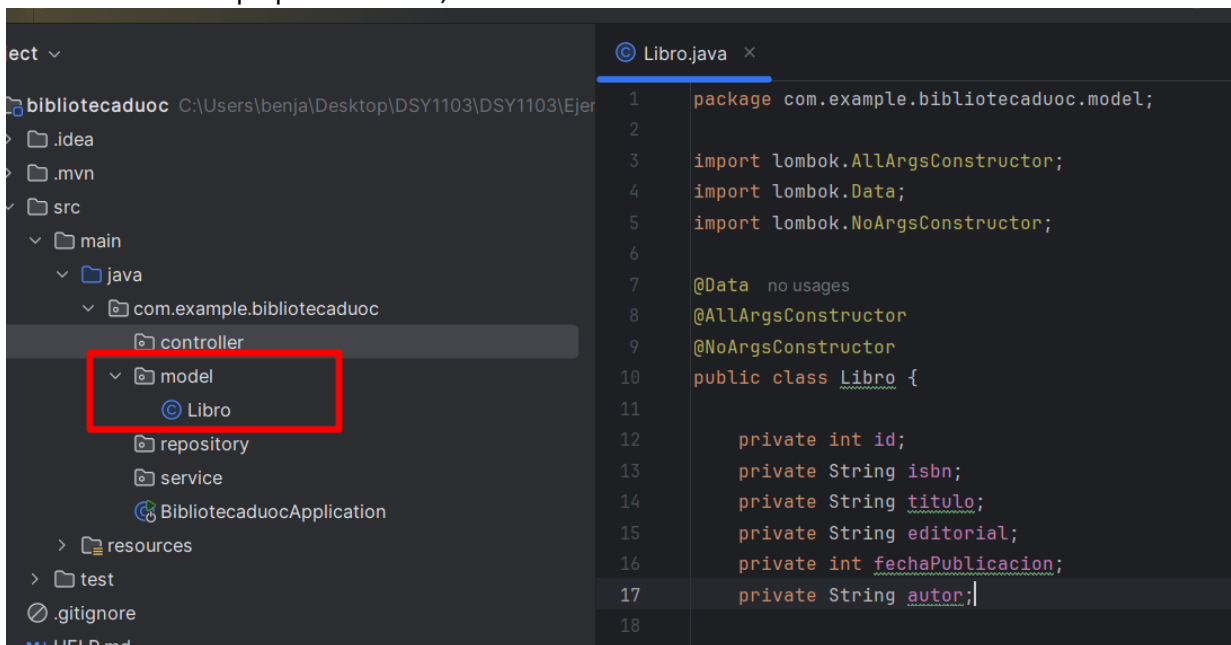
- Dentro de tu proyecto en el IDE, navega a `src/main/java/com/ejemplo/bibliotecaduoc/`
 - Crea nuevos paquetes (carpetas)
 - **controller**
 - **services**
 - **repository**
 - **model**





Paso 6: Añadiendo modelo

- Dentro del paquete **modelo**, crea una nueva clase llamada **Libro**.



@Data

La anotación @Data es una combinación de varias anotaciones de Lombok.

Genera automáticamente los siguientes métodos para la clase:

1. **@Getter:** Genera métodos getter para todos los campos.
2. **@Setter:** Genera métodos setter para todos los campos.
3. **@ToString:** Genera un método toString que incluye todos los campos.
4. **@EqualsAndHashCode:** Genera métodos equals y hashCode que comparan los campos.
5. **@RequiredArgsConstructor:** Genera un constructor con los campos finales (final) y los campos marcados con **@NonNull**.

@AllArgsConstructor

La anotación @AllArgsConstructor genera automáticamente un constructor que incluye todos los campos de la clase como parámetros. En este caso, el constructor generado sería:

Ejemplo:

```
info.java
1 public Libro(int id, String isbn, String titulo, String editorial, int fechaPublicacion, String autor) {
2     this.id = id;
3     this.isbn = isbn;
4     this.titulo = titulo;
5     this.editorial = editorial;
6     this.fechaPublicacion = fechaPublicacion;
7     this.autor = autor;
8 }
9
10
11
```

@NoArgsConstructor

La anotación @NoArgsConstructor genera automáticamente un constructor sin argumentos. Este constructor es útil cuando necesitas crear instancias de la clase sin inicializar sus campos de inmediato. El constructor generado sería:

```
info.java
1 public Libro() {
2
3     // Constructor vacío
4
5 }
```

- Resultado final
 - Ver archivo **Libro.java**

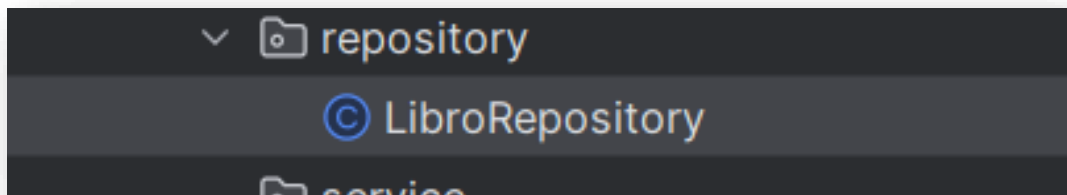
```
Libro.java x
1 package com.example.bibliotecaduoc.model;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6
7
8 @Data //Genera getters, setters, toString, equals, hashCode y un constructor con los campos requeridos. no usages
9 @AllArgsConstructor // Genera un constructor con todos los campos.
10 @NoArgsConstructor // Genera un constructor con todos los campos.
11 public class Libro {
12
13     private int id;
14     private String isbn;
15     private String titulo;
16     private String editorial;
17     private int fechaPublicacion;
18     private String autor;
19
20 }
21
```

Paso 7: Añadiendo repository

- Dentro del paquete **repository**, crea una nueva clase llamada **LibroRepository**.

```

Libro.java x
1 package com.example.bibliotecaduoc.model;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6
7
8 @Data //Genera getters, setters, toString, equals, hashCode y un constructor con los campos requeridos. no usages
9 @AllArgsConstructor // Genera un constructor con todos los campos.
10 @NoArgsConstructor // Genera un constructor con todos los campos.
11 public class Libro {
12
13     private int id;
14     private String isbn;
15     private String titulo;
16     private String editorial;
17     private int fechaPublicacion;
18     private String autor;
19
20 }
21
  
```



- Usaremos **LibroRepository** como clase ya que no lo conectaremos a una base de datos.
- Desarrollar un **CRUD** de arreglos en LibroRepository
 - Añadir **@Repository** a la clase
 - Variable privada de tipo Lista para almacenar todos los libros, se declara como arraylist vacío: *listaLibros*
 - Método que retorna la lista de libros: *obtenerLibros*


```
1 package com.example.bibliotecaduoc.repository;
2
3 import com.example.bibliotecaduoc.model.Libro;
4 import org.springframework.stereotype.Repository;
5
6 import java.util.ArrayList;
7 import java.util.List;
8
9 @Repository
10 public class LibroRepository {
11
12     // Arreglo que guardara todos los libros
13     private List<Libro> listaLibros = new ArrayList<>(); 12 usages
14
15
16     // Metodo que retorna todaa los libros
17     public List<Libro> obtenerLibros() { 1 usage
18         return listaLibros;
19     }
20 }
```

- Método que busca el libro por id, retorna libro: **buscarPorId**
- Método que busca el libro por isbn, retorna libro: **buscarPorIsbn**

```
20
21 // Buscar un libro por su id
22 public Libro buscarPorId(int id) { 2 usages
23     for (Libro libro : listaLibros) {
24         if (libro.getId() == id) {
25             return libro;
26         }
27     }
28     return null;
29 }
30
31 // Buscar un libro por su isbn
32 public Libro buscarPorIsbn(String isbn) { no usages
33     for (Libro libro : listaLibros) {
34         if (libro.getIsbn().equals(isbn)) {
35             return libro;
36         }
37     }
38     return null;
39 }
40
```

- Método que guarda el libro en el arreglo y retorna libro: **guardar**
- Método que actualiza el libro por el id, retorna libro actualizado: **actualizar**

```
41 public Libro guardar(Libro lib) { 1 usage
42     listaLibros.add(lib);
43     return lib;
44 }
45
46 public Libro actualizar(Libro lib) { 1 usage
47     int id = 0;
48     int idPosicion = 0;
49
50     for (int i = 0; i < listaLibros.size(); i++) {
51         if (listaLibros.get(i).getId() == lib.getId()) {
52             id = lib.getId();
53             idPosicion = i;
54         }
55     }
56
57     Libro libro1 = new Libro();
58     libro1.setId(id);
59     libro1.setTitulo(lib.getTitulo());
60     libro1.setAutor(lib.getAutor());
61     libro1.setFechaPublicacion(lib.getFechaPublicacion());
62     libro1.setEditorial(lib.getEditorial());
63     libro1.setIsbn(lib.getIsbn());
64
65     listaLibros.set(idPosicion, libro1);
66     return libro1;
67 }
```

- Método que elimina el libro mediante su id : **eliminar**
 - Los tres métodos son utilizados para eliminar un ítem en un arreglo.

```

76
77     public void eliminar(int id) { 1 usage
78         // alternativa 1
79         Libro libro = buscarPorId(id);
80         if (libro != null) {
81             listaLibros.remove(libro);
82         }
83
84         // alternativa 2
85         int idPosicion = 0;
86         for (int i = 0; i < listaLibros.size(); i++) {
87             if (listaLibros.get(i).getId() == id) {
88                 idPosicion = i;
89                 break;
90             }
91         }
92         if (idPosicion > 0) {
93             listaLibros.remove(idPosicion);
94         }
95
96         // otra alternativa
97         listaLibros.removeIf(x -> x.getId() == id);
98     }
99

```

La anotación **@Repository** en Spring es una especialización de la anotación **@Component**, que se utiliza para indicar que la clase es un "bean" de Spring y que debe ser gestionada por el contenedor de Spring. Más específicamente, **@Repository** se usa para anotar clases que acceden a la base de datos, encapsulan el almacenamiento, recuperación y búsqueda de objetos de dominio.

Propósito de @Repository

Identificación semántica:

- Indica que la clase es un repositorio, una abstracción que se encarga de la persistencia de datos.
- Hace que el propósito de la clase sea más claro y mejora la legibilidad del código.

Manejo de excepciones:

- Spring proporciona una traducción automática de excepciones específicas de acceso a datos (como las excepciones de JDBC) a excepciones de Spring de nivel superior y no verificadas.
- La anotación **@Repository** permite que Spring traduzca automáticamente las excepciones lanzadas en la capa de persistencia.

Gestión de beans:

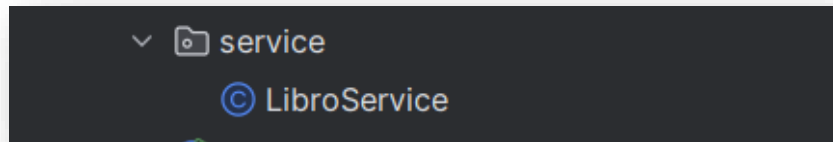
Las clases anotadas con **@Repository** son detectadas automáticamente por el escaneo de componentes de Spring y se convierten en beans gestionados por el contenedor de Spring.

Permite la inyección de dependencias en otras partes de la aplicación.

En LibroService vamos a ver como esto funciona

Paso 8: Añadiendo service

- Dentro del paquete **service**, crea una nueva clase llamada **LibroService**.



- Añadir **@Service** a la clase
- Variable privada de tipo **LibroRepository**, añadir la **@anotacion** de **@Autowired**
- Añadir todos los métodos del Repositorio

```

Libro.java  LibroRepository.java  LibroController.java  LibroService.java x
1
2
3  import com.example.bibliotecaduoc.model.Libro;
4  import com.example.bibliotecaduoc.repository.LibroRepository;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.stereotype.Service;
7  import java.util.List;
8
9  @Service 2 usages
10 public class LibroService {
11     @Autowired
12     private LibroRepository libroRepository;
13
14     public List<Libro> getLibros() { 1 usage
15         return libroRepository.obtenerLibros();
16     }
17
18     public Libro saveLibro(Libro libro) { 1 usage
19         return libroRepository.guardar(libro);
20     }
21
22     public Libro getLibroId(int id) { 1 usage
23         return libroRepository.buscarPorId(id);
24     }
25
26     public Libro updateLibro(Libro libro) { 1 usage
27         return libroRepository.actualizar(libro);
28     }
29
30     public String deleteLibro(int id) { 1 usage
31         libroRepository.eliminar(id);
32         return "producto eliminado";
33     }
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

@Service

Propósito

@Service es una especialización de @Component que se usa para definir una clase como un servicio en la capa de negocio. Indica que una clase se utiliza para contener la lógica de negocio y debería ser gestionada por el contenedor de Spring.

Uso

Anotando una clase con `@Service`, se hace que Spring la detecte automáticamente durante el escaneo de componentes y la registre como un **bean** gestionado. Esto facilita la organización y la separación de las diferentes capas de una aplicación (como la capa de persistencia, la capa de servicio, y la capa de presentación).

@Autowired

Propósito

`@Autowired` es una anotación que permite la inyección automática de dependencias en Spring. Con esta anotación, Spring puede resolver e inyectar automáticamente las dependencias de un bean.

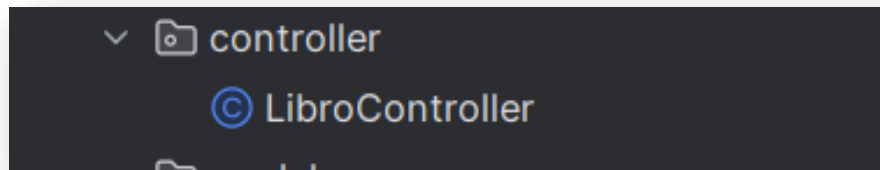
Uso

`@Autowired` se puede usar en: Campos, Constructores, Métodos de establecimiento (setters)

Spring resolverá e inyectará la dependencia adecuada del contexto de la aplicación.

Paso 9: Añadiendo controller

- Dentro del paquete **controller**, crea una nueva clase llamada **LibroController**.



- Añadir **@Service** a la clase
- Variable privada de tipo **LibroRepository**, añadir la **@anotacion** de **@Autowired**
- Añadir todos los métodos del Repositorio

```

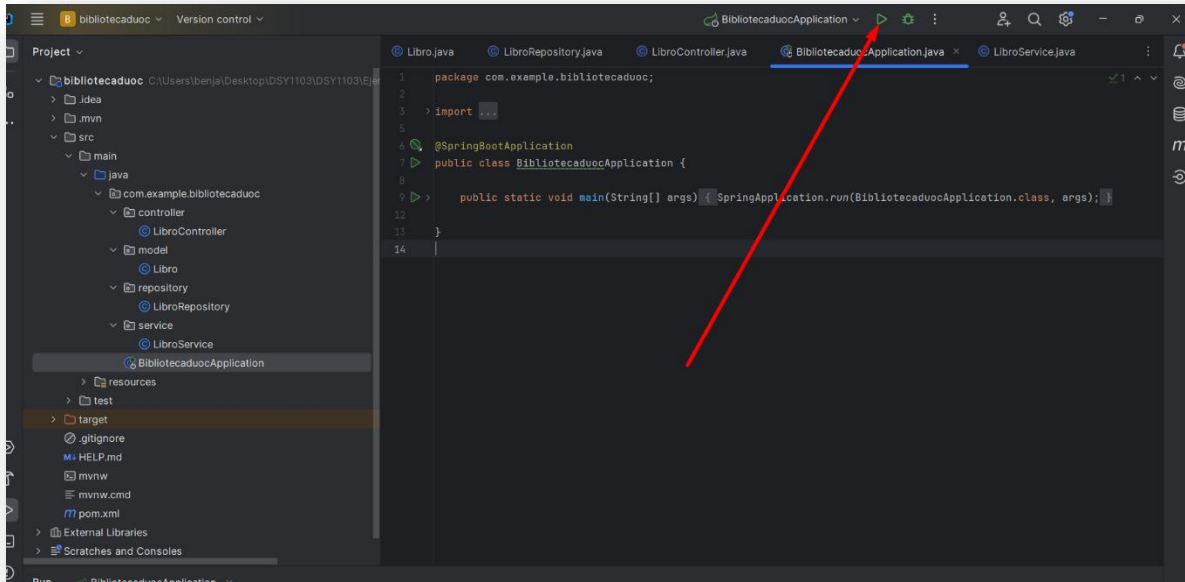
Libro.java  LibroRepository.java  LibroController.java  LibroService.java
1  package com.example.bibliotecaduoc.controller;
2
3  import com.example.bibliotecaduoc.model.Libro;
4  import com.example.bibliotecaduoc.service.LibroService;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.web.bind.annotation.*;
7
8  import java.util.List;
9
10 @RestController
11 @RequestMapping("/api/v1/libros")
12 public class LibroController {
13
14     @Autowired
15     private LibroService libroService;
16
17     @GetMapping
18     public List<Libro> listarLibros() {
19         return libroService.getLibros();
20     }
21
22     @PostMapping
23     public Libro agregarLibro(@RequestBody Libro libro) {
24         return libroService.saveLibro(libro);
25     }
26
27     @GetMapping("/{id}")
28     public Libro buscarLibro(@PathVariable int id){
29         return libroService.getLibroId(id);
30     }
31
32     @PutMapping("/{id}")
33     public Libro actualizarLibro(@PathVariable int id, @RequestBody Libro libro){
34         // el id lo usaremos mas adelante
35         return libroService.updateLibro(libro);
36     }
37
38     @DeleteMapping("/{id}")
39     public String eliminarLibro(@PathVariable int id) {
40         return libroService.deleteLibro(id);
41     }
42 }

```

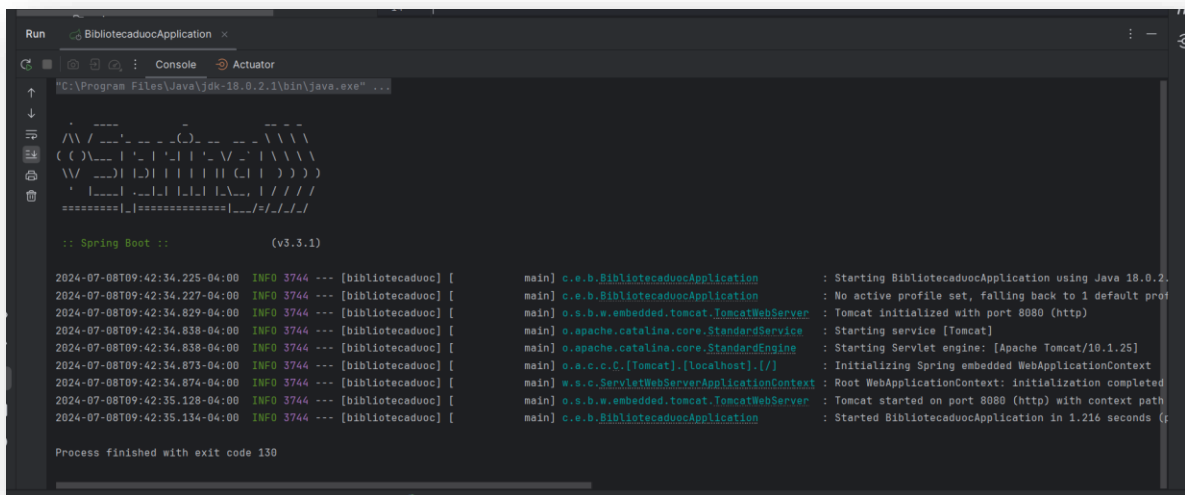
Nota: El método PUT pide un id como parámetro, esto lo usaremos más adelante.

Paso 10: Ejecutar la Aplicación

- Dentro del IDE, busca la clase **BibliotecaduocApplication** en el paquete principal y ejecútala como una aplicación Java.



Consola con resultados correctos



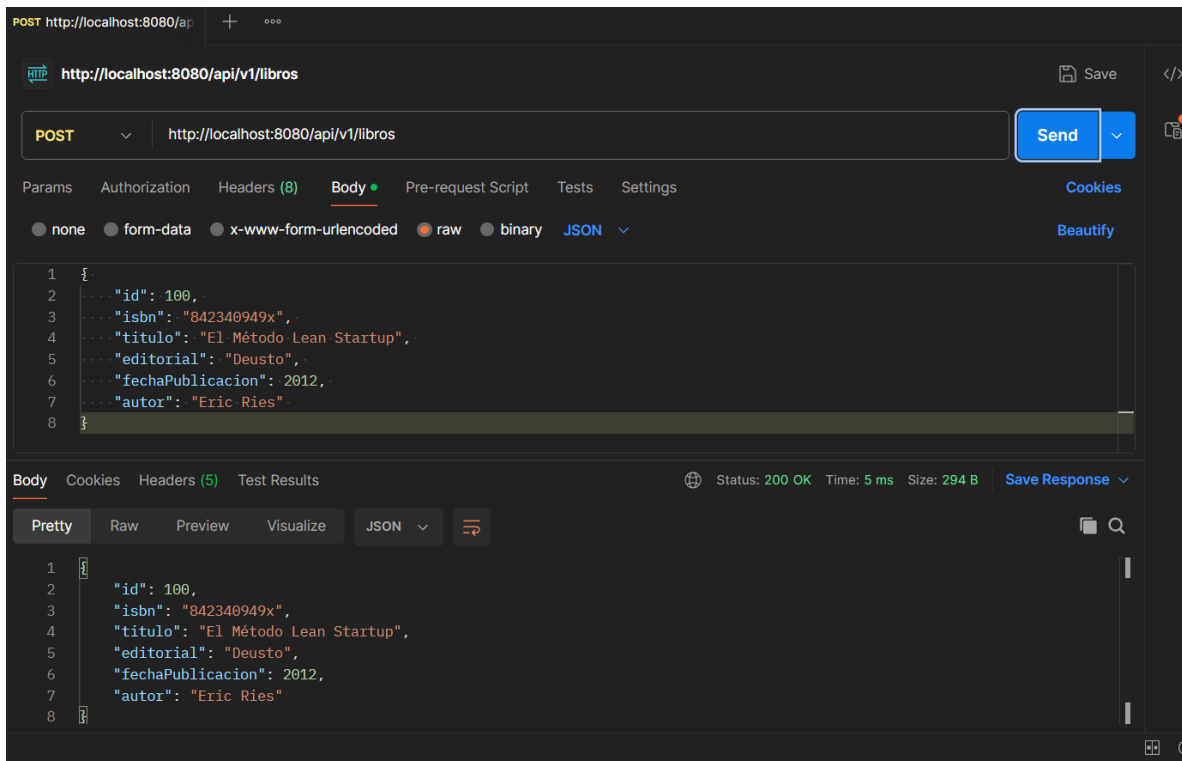
Paso 11: Probar la aplicación en POSTMAN

CREAR LIBRO

- Método: **POST**
- URL: <http://localhost:8080/api/v1/libros>
- Body: marcar raw
 - Parámetro de entrada JSON

Parámetros en JSON

```
{  
  "id": 100,  
  "isbn": "842340949x",  
  "titulo": "El Método Lean Startup",  
  "editorial": "Deusto",  
  "fechaPublicacion": 2012,  
  "autor": "Eric Ries"  
}
```

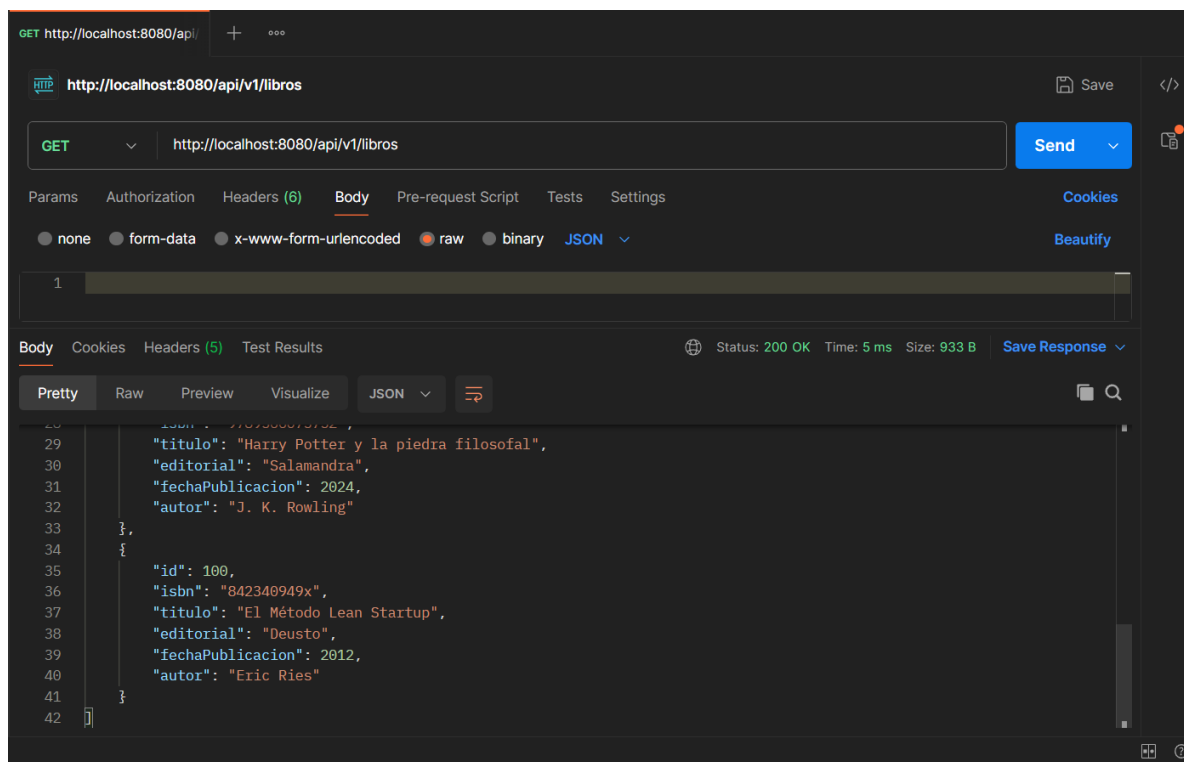


Añade los cuatro libros restantes

ISBN	TITULO	EDITORIAL	FECHA PUBLICACION	AUTOR
9789569646638	Fuego y Sangre	Penguin Random House Grupo Editorial	2018	George R. R. Martin
9789563494150	Quique Hache: El Mall Embrujado y Otras Historias	Sm Ediciones	2014	Sergio Gomez
9781484256251	Spring Boot Persistence Best Practices	Apress	2020	Anghel Leonard
9789566075752	Harry Potter y la piedra filosofal	Salamandra	2024	J. K. Rowling

VER TODOS LOS LIBRO

- Método: **GET**
- URL: <http://localhost:8080/api/v1/libros>



The screenshot shows a REST client interface with the following details:

- Request:** Method **GET**, URL **http://localhost:8080/api/v1/libros**.
- Response:** Status **200 OK**, Time **5 ms**, Size **933 B**.
- Body (JSON):**

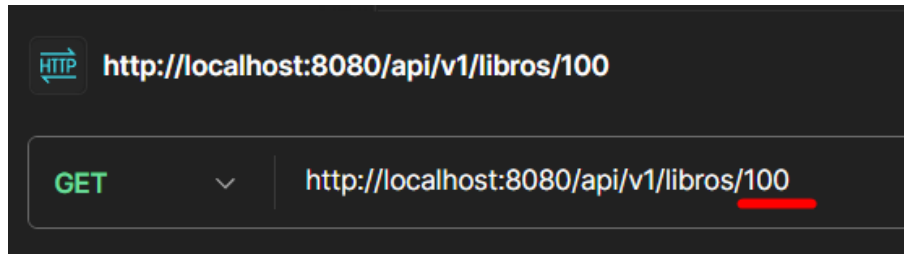
```

[
  {
    "id": 100,
    "isbn": "842340949x",
    "titulo": "El Método Lean Startup",
    "editorial": "Deusto",
    "fechaPublicacion": 2012,
    "autor": "Eric Ries"
  },
  {
    "id": 101,
    "isbn": "9789566075752",
    "titulo": "Harry Potter y la piedra filosofal",
    "editorial": "Salamandra",
    "fechaPublicacion": 2024,
    "autor": "J. K. Rowling"
  }
]

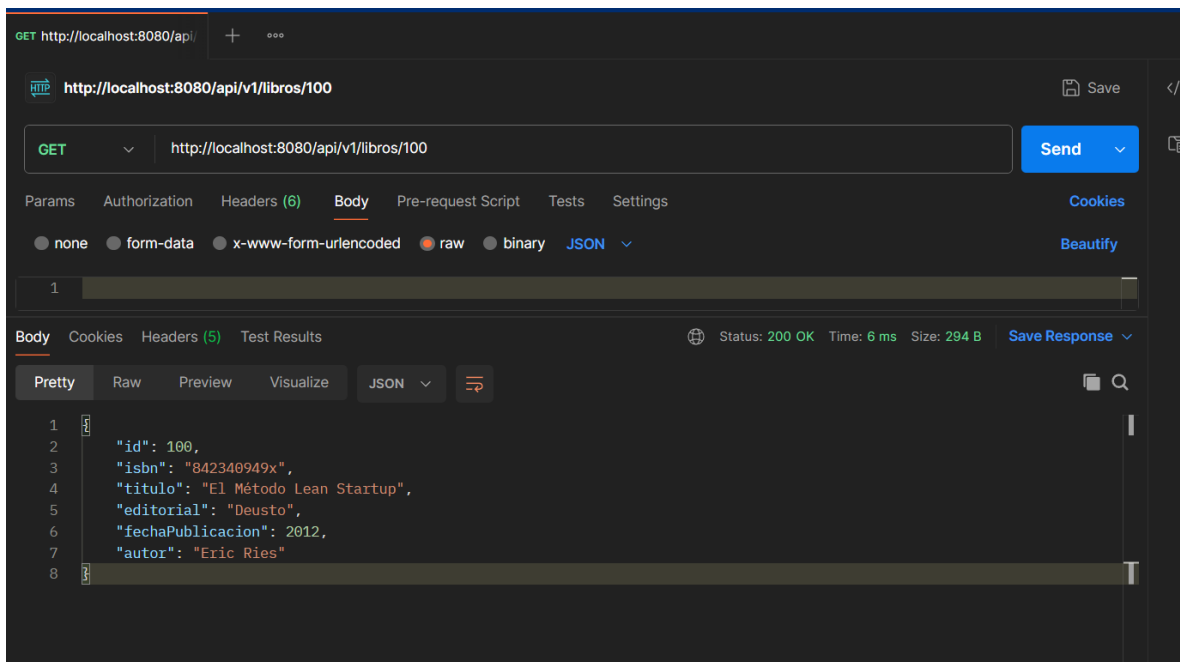
```

BUSCAR UN LIBRO

- Método: **GET**
- URL: <http://localhost:8080/api/v1/libros/{id}>



La búsqueda se relaciona con la entrega del recurso **100**



ACTUALIZAR UN LIBRO

- Método: **PUT**
- URL: <http://localhost:8080/api/v1/libros/{id}>
 - Body: marcar raw
 - Parámetro de entrada JSON

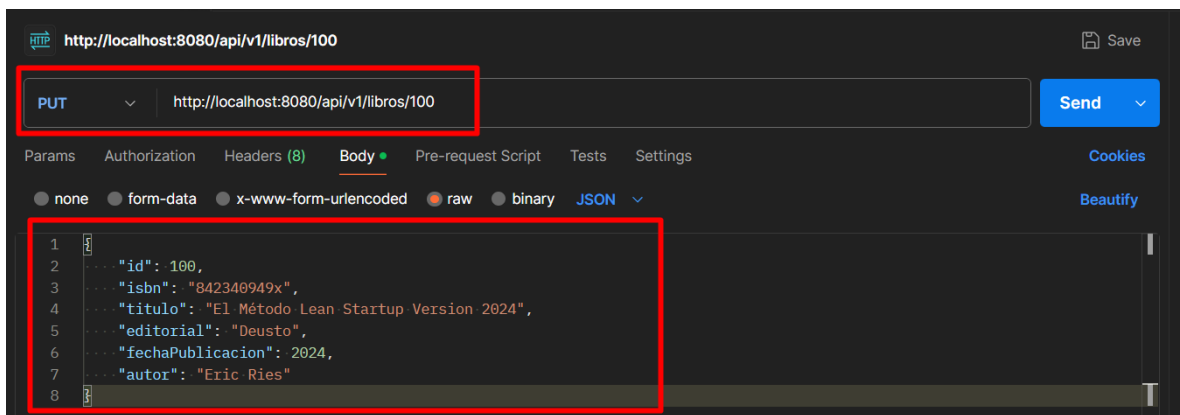
Actualmente

```
1 {
2   "id": 100,
3   "isbn": "842340949x",
4   "titulo": "El Método Lean Startup",
5   "editorial": "Deusto",
6   "fechaPublicacion": 2012,
7   "autor": "Eric Ries"
8 }
```

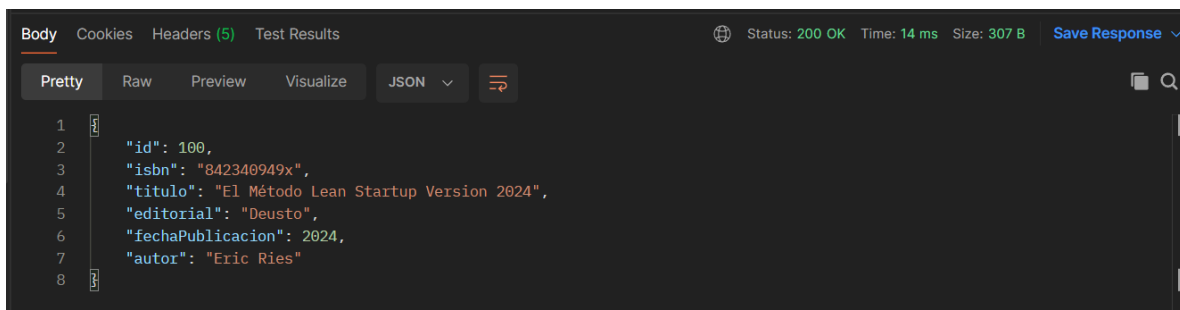
Actualización

```
1 {
2   ... "id": 100,
3   ... "isbn": "842340949x",
4   ... "titulo": "El Método Lean Startup Version
5   ... "editorial": "Deusto",
6   ... "fechaPublicacion": 2024,
7   ... "autor": "Eric Ries"
8 }
```

Envío de solicitud put

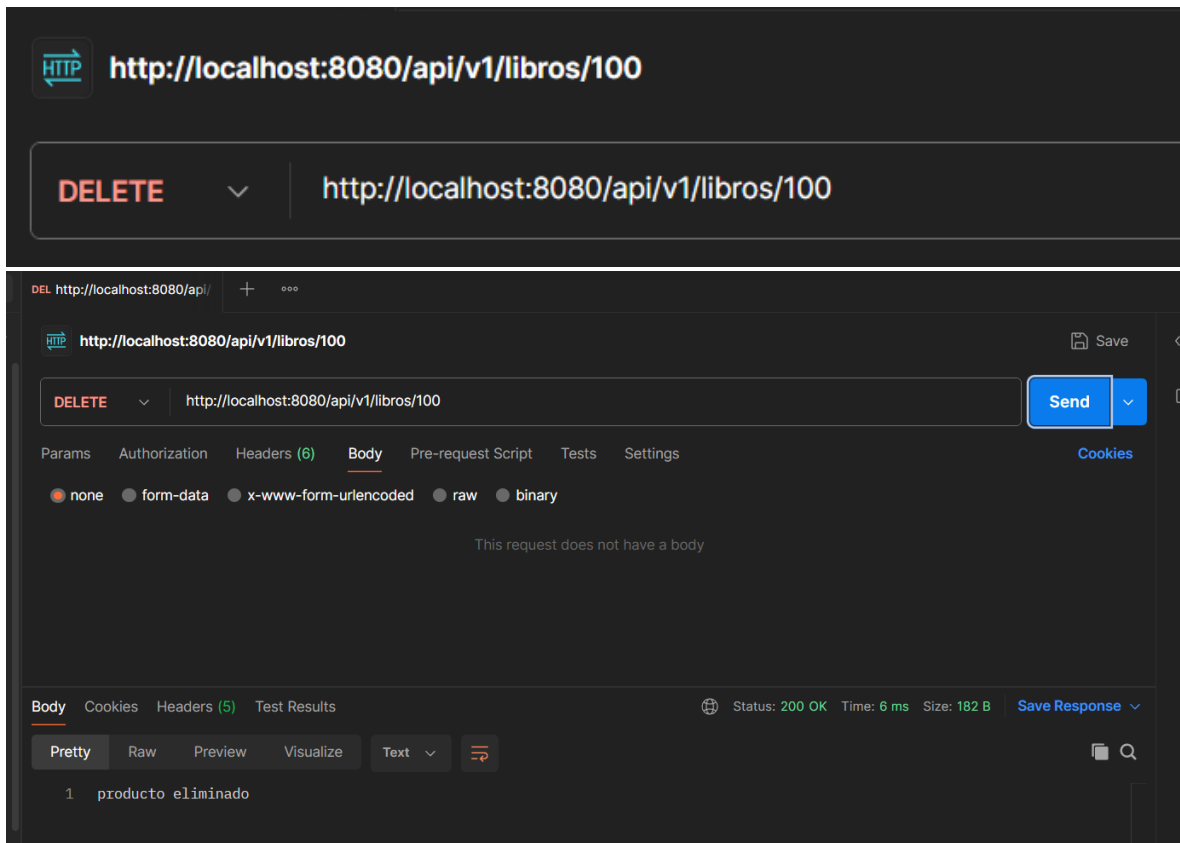


Respuesta



ELIMINAR UN LIBRO

- Método: **DELETE**
- URL: <http://localhost:8080/api/v1/libros/{id}>



!!! 🐶 MUY BIEN 🐶 !!!

Has mejorado tu proyecto con funcionalidades de spring boot, y has implementado endpoint para poder guardar, buscar, actualizar y eliminar una lista de libros.

Estás listo para seguir explorando más funcionalidades y conectar tu aplicación a una base de datos.

